Bags, Vectors & Transformers

Transformers in Practice · Day 3, Part 2

Denise J. Roth, PhD Candidate
Strategic Communication Group · Wageningen University & Research

From Theory to Practice

We know *what* transformers are and *how* attention works. Now the questions that matter for actually **doing research** with them:

- The **pre-train then fine-tune** paradigm
- Three ways to **use** a pre-trained model, and how **zero-shot** and **fine-tuning** work
- Picking the right approach for **your project**
- What researchers in our fields **use them for**
- Practical **parameters** and model **variants**, including Dutch
- How to **measure success**

Then notebook Part B, together on screen.

The Big Idea: Pre-train Once, Fine-tune Often

The reason transformers are usable by *everyone*, not just big labs, is a two-stage split:



The model arrives already knowing **how language works** (grammar, meaning, world knowledge). You just **nudge** it toward your specific task with a modest amount of labeled examples. This is called **transfer learning**.

Why Fine-tuning Is a Big Deal for Social Science

The old way

- Train a model **from scratch**
- Needs **huge** labeled datasets
- Out of reach for most projects
- Lots of compute

With fine-tuning

- Start from a **pre-trained** model
- A **few hundred to a few thousand** labeled examples can suffice
- Runs on a **single GPU** (free in Colab)
- Often beats classic methods

Hand-labeling 500 documents is a realistic afternoon. Hand-labeling a million is not. Fine-tuning meets you where your data is.

Three Ways to Use a Pre-trained Model

1. Off-the-shelf

Use an existing fine-tuned model as-is.

No training. E.g. a ready-made sentiment or topic classifier.

Fastest; only if a model for your task exists.

2. Frozen features

Extract **contextual embeddings**, feed them to a simple classifier.

Model stays **frozen**; you train only the classifier.

Like Day 2, but context-aware vectors.

3. Fine-tuning

Update the **whole model** on your labeled task.

Most powerful, needs a GPU and labels.

Best performance when you have training data.

Increasing effort, increasing power — left to right. Pick the lightest one that solves your problem.

How Zero-Shot Classification Works

How it was trained

Hundreds of thousands of sentence pairs, such as:

Text: A man is playing a guitar on stage.

Claim: Someone is making music.

→ **follows** (other pairs: contradicts, or neither)

How you use it

Text: A bill to expand access to affordable health insurance coverage.

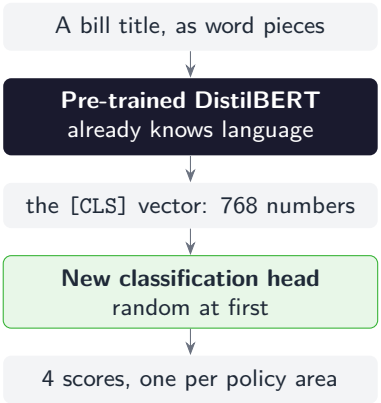
One claim per label:

✔ This bill is about **health**.
This bill is about education.
This bill is about taxation.
This bill is about armed forces and national security.

The label whose claim follows most clearly wins.

No labels needed, but the wording matters, it's slow, and fine-tuning usually wins.

How Fine-tuning Works



Training, round after round

1. Feed a **batch** of labeled titles (16 at a time).
2. Compare the predictions with the true labels: the **loss**.
3. Nudge **all** the weights a little to reduce it (learning rate 2e-5).
4. After each pass through the data (an **epoch**), check the validation set.
5. Stop after 2 to 4 epochs. Use the test set **once**, at the end.

The warning about **newly initialized weights** that you'll see in the notebook is this new head. It's expected.

Pick the Approach

IN PAIRS Off-the-shelf, frozen features, fine-tuning, or no BERT at all? Two minutes.

50,000 Dutch news articles. You want the ones about farmers' protests. Nothing is labeled yet.

English product reviews. A sentiment model trained on reviews is already on the Hub.

800 hand-coded speeches. You coded them for populism. 20,000 more are waiting.

2,000 open survey answers. You want to see which themes come up. There are no categories yet.

Pick the Approach

IN PAIRS Off-the-shelf, frozen features, fine-tuning, or no BERT at all? Two minutes.

50,000 Dutch news articles. You want the ones about farmers' protests. Nothing is labeled yet.

→ Start cheap: keywords or zero-shot to pre-sort, hand-label a sample to check, then fine-tune a Dutch model if needed.

800 hand-coded speeches. You coded them for populism. 20,000 more are waiting.

→ Fine-tuning: this is what it's for. Keep a test set aside that the model never sees.

English product reviews. A sentiment model trained on reviews is already on the Hub.

→ Off-the-shelf, but only after checking it on a hundred or two of your own texts. Validation is never optional.

2,000 open survey answers. You want to see which themes come up. There are no categories yet.

→ No classifier at all: embed the answers and cluster them. That's Thursday, with Art.

What Can You Use Them For?

Classification tasks

- **Sentiment** & stance detection
- Topic / policy-area coding
- Detecting **incivility**, framing, emotion
- Scaling up hand-coding to a full corpus

Beyond classification

- **Semantic search** & document similarity
- Clustering by meaning
- Named-entity recognition
- Measuring meaning, like Day 2 — but context-aware

The common thread: any task where you would otherwise hand-read thousands of documents.

BERT in Published Social Science

Less annotating, more classifying

Models trained on inference tasks (today's zero-shot trick) do well with far fewer hand-coded examples.

Laurer, van Atteveldt, Casas & Welbers (2024), *Political Analysis*

Sentiment is not stance

A negative tone doesn't tell you whether someone is for or against something. Measure stance directly, with classifiers that know the target.

Bestvater & Monroe (2023), *Political Analysis*

Emotions in German political text

Dictionaries, word embeddings and a transformer compared for measuring discrete emotions: Days 1 to 3 in one paper.

Widmann & Wich (2023), *Political Analysis*

PopBERT

Detects populist statements, and the left- or right-wing ideas they come with, in German Bundestag speeches.

Erhard, Hanke, Remer, Falenska & Heiberger (2024), *Political Analysis*

The common recipe: hand-code a sample, train, validate, then measure at scale.

Practical Parameters to Know

Things that bite you

· **Sequence length**: BERT caps at **512 tokens** — long documents must be split

· **Model size**: more parameters = better but slower

· **GPU**: fine-tuning really wants one

· **Batch size** & **learning rate**: the main knobs when fine-tuning

Sensible defaults

· Start with a **small model** (DistilBERT) to prototype

· 2–4 fine-tuning **epochs** is usually enough

· Learning rate around **2e-5**

· Always keep a **held-out test set** (Day 1 discipline!)

A Family of Models

"BERT" is really a whole family. You rarely need the original — pick the variant that fits your constraints.

BERT	the original; solid general-purpose baseline
RoBERTa	better-trained BERT; often higher accuracy
DistilBERT	smaller & faster (∼40% lighter), minimal accuracy loss — great for prototyping
mBERT / XLM-R	**multilingual**; one model, 100+ languages
BERTje / RobBERT	**Dutch** models, trained on Dutch text

*For Dutch social-science data, **BERTje** or **RobBERT** will usually beat a multilingual model — and both are a click away.*

Measuring Success

- ✔ **Accuracy**: the share of texts labeled correctly.
- ✔ **Precision** (per label): of the texts the model put in this label, how many belong there?
- ✔ **Recall** (per label): of the texts that belong to this label, how many did the model find?
- ✔ **F1**: the balance of the two. **Macro-F1**: the average F1 over all labels, so a rare label counts as much as a common one.

Why accuracy can fool you

100 sentences: 90 *no claim*, 10 *claim*. A lazy model answers *no claim* every time.

Accuracy: **0.90**
Recall for *claim*: **0**
Macro-F1: **0.47**

The confusion matrix: rows are the true labels, columns the predictions. The diagonal holds the correct answers; everything else shows *which* labels get mixed up. Always compare with a **baseline**, and look at the scores per label.

One Task, Four Ways

Notebook Part B: **2,000 US bill titles**, four policy areas (Health, Education, Taxation, Armed Forces & National Security), 1,400 of them for training.

TF-IDF + logistic regression

Day 1's baseline. Seconds, no GPU.

Zero-shot

No labels at all: does "This bill is about health" fit?

Frozen BERT + logistic regression

BERT's vectors, a simple classifier on top.

Fine-tuned DistilBERT

The whole model learns the task.

YOUR TURN **Place your bets.** Which one wins, and by how much? Write down a macro-F1 between 0 and 1 for each.

We'll compare your bets with the results in notebook Section 12.

Practice: The Takeaways

- ✔ **Pre-train once, fine-tune often**: the model already knows language; you adapt it cheaply
- ✔ Three usage modes: **off-the-shelf** (zero-shot included), **frozen features**, **fine-tuning**. Pick the lightest that works
- ✔ Zero-shot needs no labels but depends on wording; fine-tuning trains a **new head** on your labels
- ✔ Mind the **512-token limit**, model size, and the need for a GPU when fine-tuning
- ✔ **DistilBERT** to prototype, **BERTje/RobBERT** for Dutch
- ✔ Judge a classifier by **macro-F1 and per-label scores**, never by accuracy alone

To the notebook, Part B, on screen: the bill titles, four approaches, and your bets.